UNIT 2:REVERSING MALICIOUS CODE

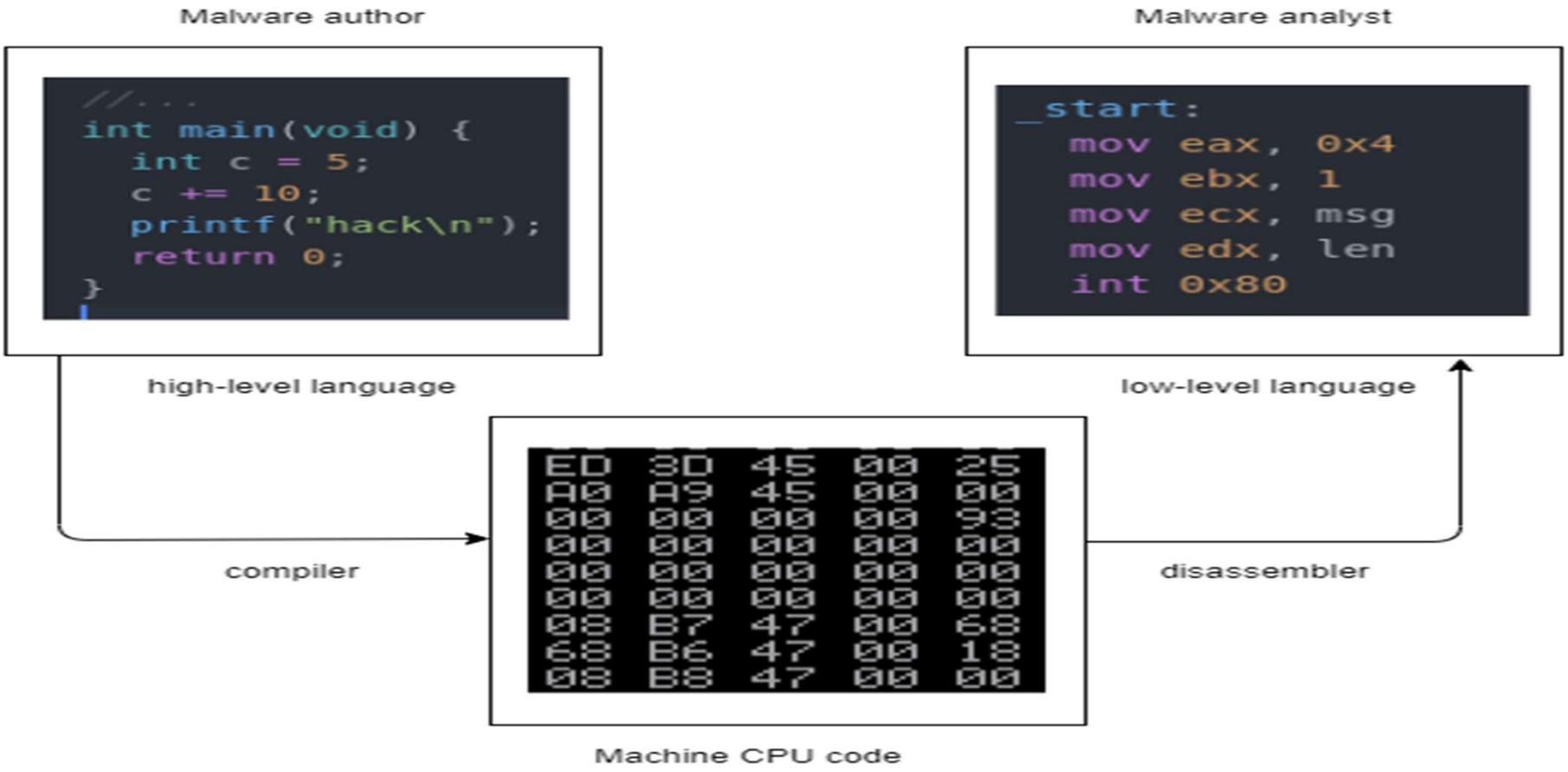
Dhanashree Kulkarni

Index

- Understanding Core x86 Assembly Concepts to Perform Malicious Code Analysis;
- Identifying Key Assembly Logic Structures with a Disassembler;
- Following Program Control Flow to Understand Decision Points During Execution;
- Recognizing Common Malware Characteristics at the Windows API Level (Registry Manipulation, Keylogging, HTTP Communications, Droppers);
- Extending Assembly Knowledge to Include x64 Code Analysis

- Any person who considers himself close to the world of information security, especially **malware analysts (blue team)** and **exploit developers (red team)**, must have a basic understanding of assembly language.
- In traditional computer architecture, a computer system can be represented as **several levels of abstraction** that create a way of hiding the implementation details.
- For simplicity, we will assume that we have three levels of coding when analyzing malware.

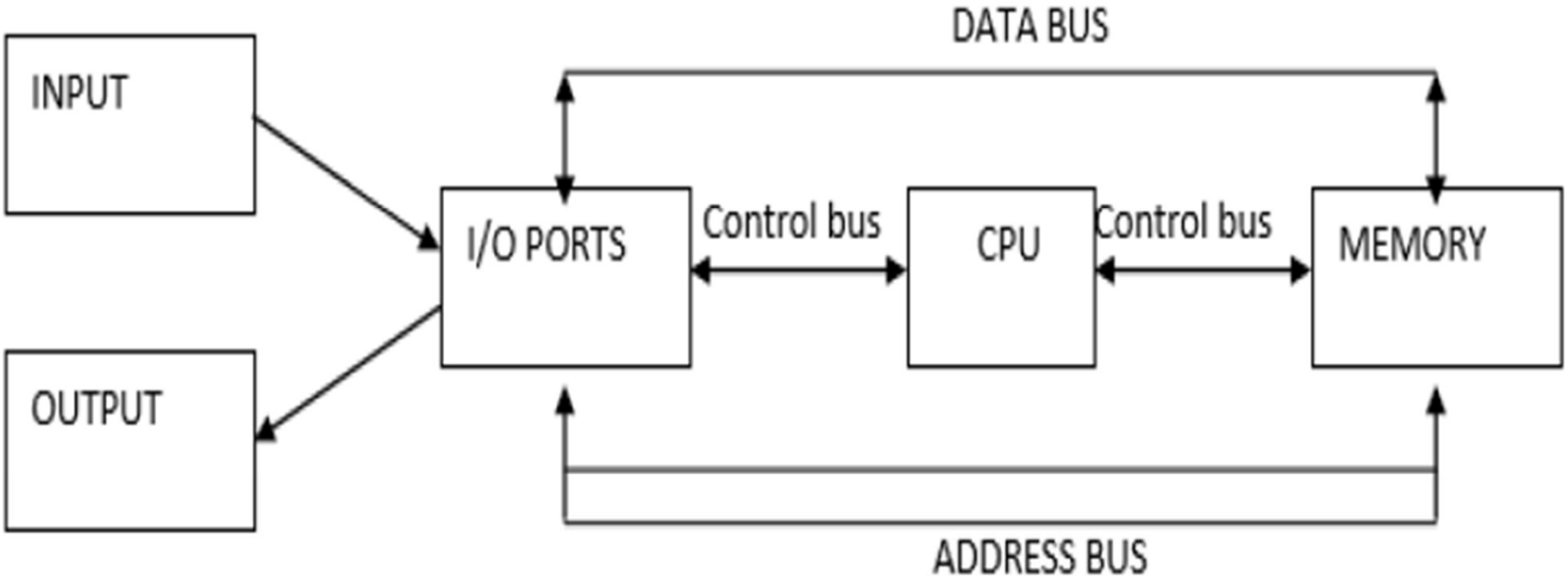
This is very simplest model, in real life computer systems are generally described with more than three levels of abstraction



- HARDWARE : the hardware level, the only physical level, consists of **electrical circuits that implement complex combinations of logical operators such as XOR, AND, OR, and NOT gates, known as digital logic.** Because of its physical nature, hardware cannot be easily manipulated by software.
- MICROCODE - also known as **firmware.** Microcode operates **only on the exact circuitry for which it was designed.** It contains **microinstructions** that **translate** from the **higher machine-code level** to provide a way to **interface with the hardware.**

- MACHINE CODE - the machine code level consists of **opcodes, hexadecimal digits that tell the processor what you want it to do.** It's not just one language called machine code. It's many different kinds of machine code. Just as we speak different languages as people, machines speak different languages.
- LOW-LEVEL LANGUAGES - a **low-level languages is a human-readable version of a computer architecture's instruction set.** The most common low-level language is **assembly language.** Assembly language which corresponds to the different architectures is by far the most important tool in any malware analyst's toolkit.

- HIGH-LEVEL LANGUAGES - Most computer programmers include who write malware, operate at the level of high-level languages. High-level languages **provide strong abstraction from the machine level and make it easy to use programming logic and flow-control mechanisms.**
- INTERPRETED LANGUAGES - Interpreted languages are at the top level. The code at this level is **not compiled into machine code**, instead, **it is translated into bytecode**. Bytecode executes within an interpreter, which is a program that translates bytecode into executable machine code on the fly at runtime. For example, python. Python is well suited for quick malware analysis. For example, a library such as **pefile**.



Block diagram of simple computer or microcomputer.

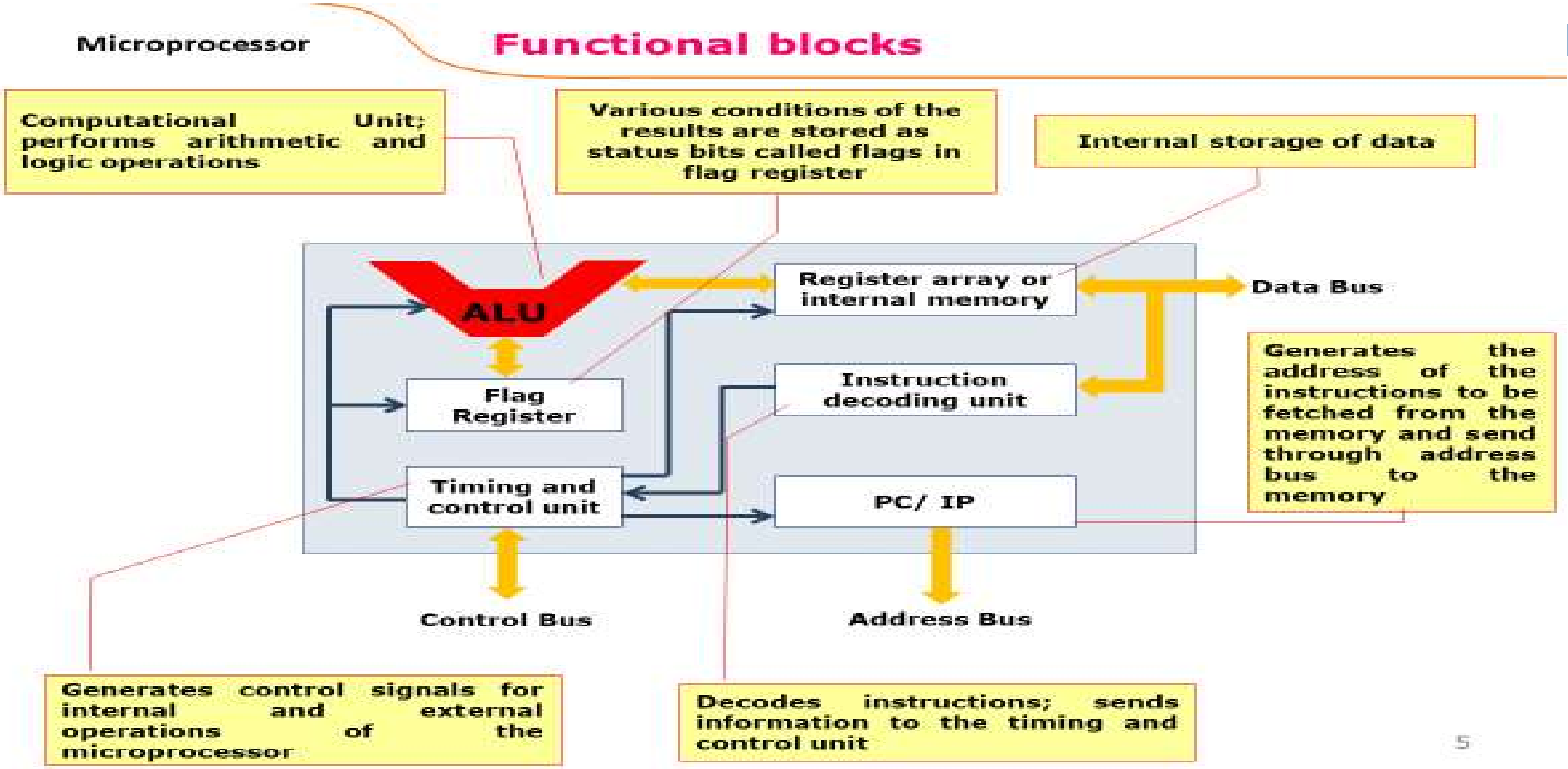
- MEMORY: The memory section usually consists of a mixture of RAM and ROM. It may also have magnetic floppy disks, magnetic hard disks, or laser optical disks. Memory has two purposes. The first purpose is to store the binary codes for the sequence of instructions you want the computer to carry out. When you write a computer program, what you are really doing is just writing a sequential list of instructions for the computer. The second purpose of the memory is to store the binary-coded data with which the computer is going to be working.

- INPUT/OUTPUT: The input/output or I/O section allows the computer to take in data from the outside world or send data to the outside world. These allow the user and the computer to communicate with each other. The actual physical devices used to interface the computer buses to external systems are often called ports.

- CPU: The central processing unit or CPU controls the operation of the computer. It fetches binary-coded instruction of the computer. It **fetches binary-coded instructions from memory, decodes the instructions into a series of simple actions, and carries out these actions.**
- The CPU contains an arithmetic logic unit, or ALU. Which can perform add, subtract, OR, AND, invert, or exclusive-OR operations on binary words when instructed to do so.
- The CPU also contains an **address counter** which is used to **hold** the **address of the next instruction** or data to be fetched from memory, general-purpose registers which are used for temporary storage of binary data, and circuitry which generates the control bus signals.

- ADDRESS BUS: The address bus consists **of 16, 20, 24, or more parallel signal lines.** On these lines the **CPU sends out the address of the memory location that is to be written to or read from.** The number of address lines determines the number of memory locations that the CPU can address. If the CPU has N address lines then it can directly address 2N memory locations.
- DATA BUS: **The data bus consists of 8, 16, 32 or more parallel signal lines.** As indicated by the double-ended arrows on the data bus line, **the data bus lines are bi-directional. This means that the CPU can read data in on these lines from memory or from a port as well as send data out on these lines to memory location or to a port.** Many devices in a system will have their outputs connected to the data bus, but the outputs of only one device at a time will be enabled.
- CONTROL BUS: **The control bus consists of 4-10 parallel signal lines.** The **CPU sends out signals on the control bus to enable the outputs of addressed memory devices or port devices.** Typical control bus signals **are memory read, memory write, I/O read, and I/O writer. To read a byte of data from a memory location, for example, the CPU sends out the address of the desired byte on the address bus and then sends out a memory read signal on the control bus.**

the x86 architecture



Block diagram of 8086

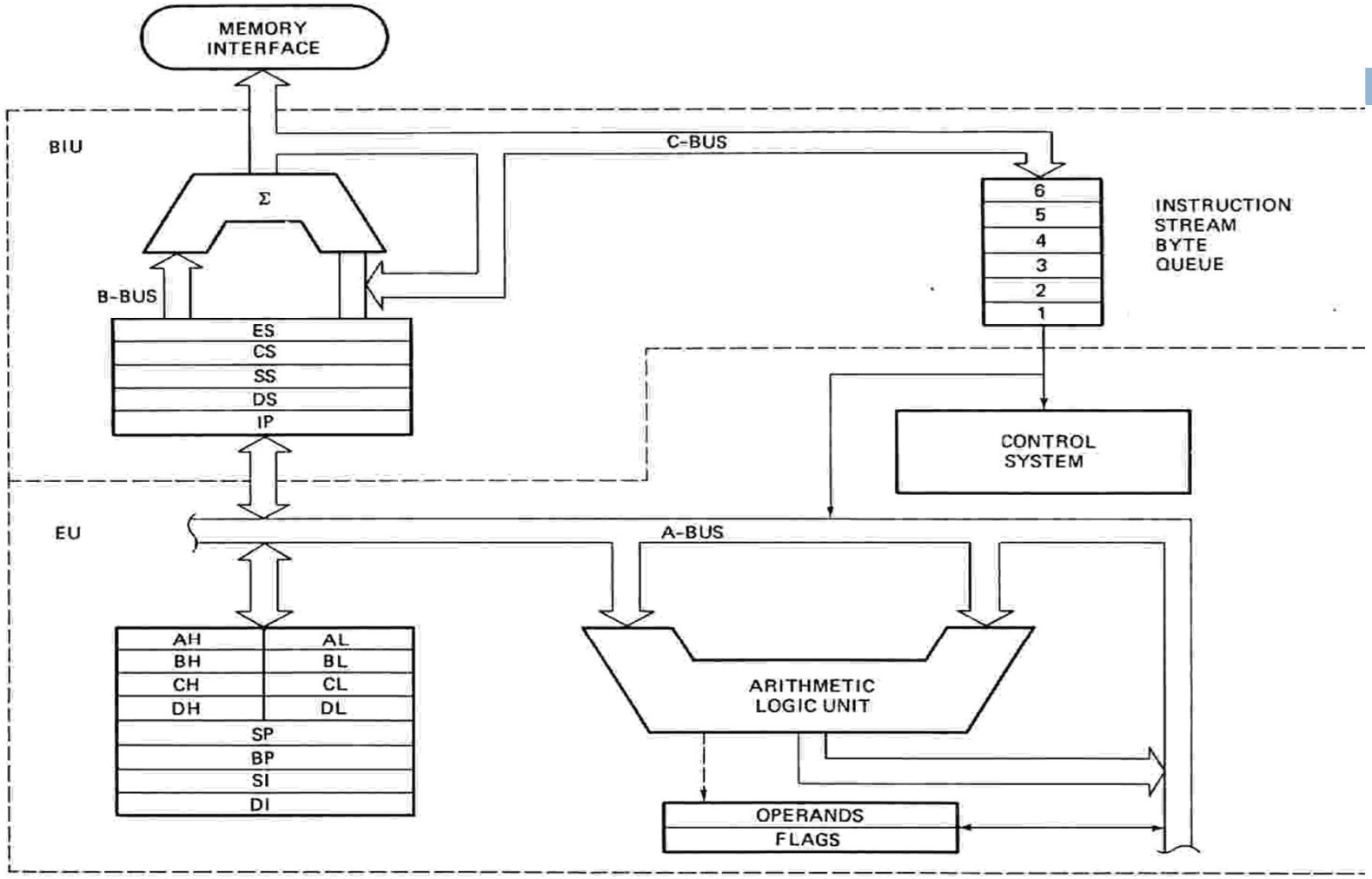


FIGURE 2-7 8086 internal block diagram. (*Intel Corp.*)

- 8086 CPU is divided into two independent functional parts
  - Bus Interface Unit(BIU)
  - Execution Unit(EU)
- Dividing the work between these two units' speeds up processing.

The Execution Unit (EU)

- The Execution Unit (EU):
  - The execution unit of the 8086 tells the BIU where to fetch instructions or data from, decodes instructions, and executes instructions.
  - The EU contains control circuitry, which directs internal operations.
  - A decoder in the EU translates instructions fetched from memory into a series of actions, which the EU carries out.
  - The EU has a 16-bit arithmetic logic unit (ALU) which can add, subtract, AND, OR, XOR, increment, decrement, complement or shift binary numbers.
- The main functions of EU are:
  - Decoding of Instructions
  - Execution of instructions
    - Steps
    - EU extracts instructions from top of queue in BIU
    - Decode the instructions
    - Generates operands if necessary
    - Passes operands to BIU & requests it to perform read or write bus cycles to memory or I/O
    - Perform the operation specified by the instruction on operands

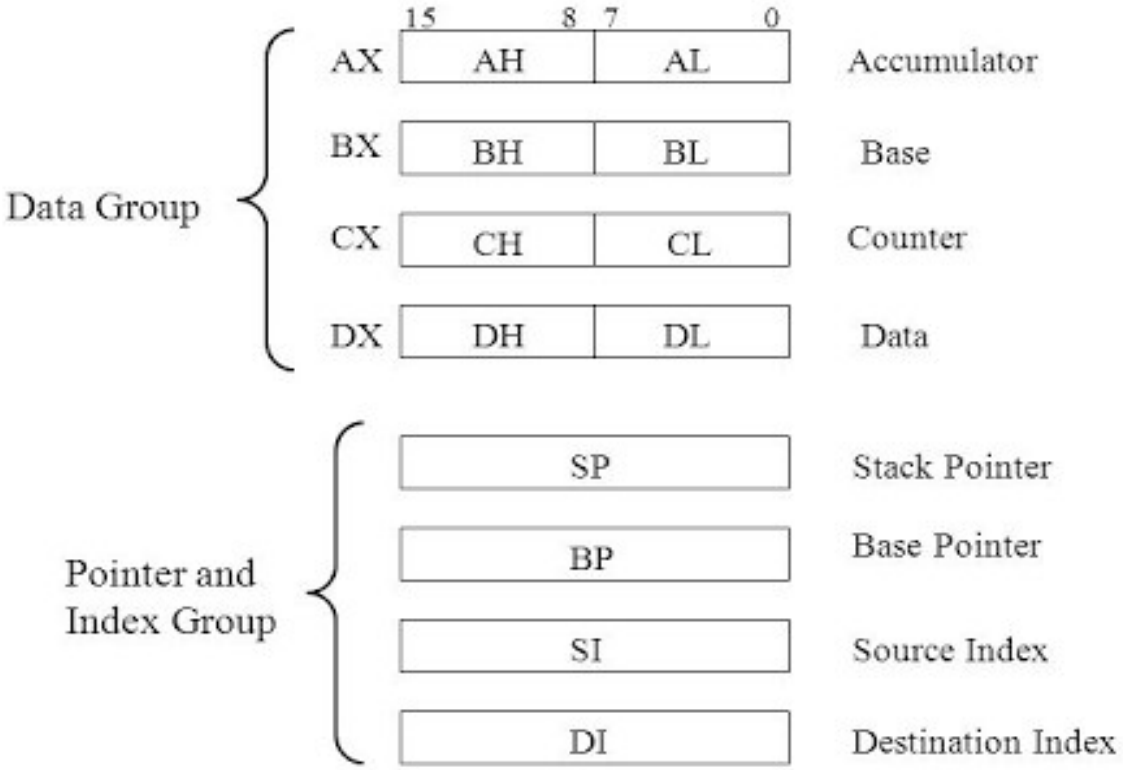
Bus Interface Unit (BIU)

- The BIU sends out addresses, fetches instructions from memory, reads data from ports and memory, and writes data to ports and memory.
- In simple words, the BIU handles **all transfers of data and addresses on the buses for the execution unit.**
- 8086 HAS PIPELINING ARCHITECTURE:
  - While the **EU is decoding an instruction or executing an instruction,** which **does not require use of the buses,** the BIU **fetches up to six instruction bytes for the following instructions.**
  - The BIU stores these **pre-fetched bytes in a first-in-first-out register set called a queue.**
  - When the EU is ready for its next instruction from the queue in the BIU. This is much faster than sending out an address to the system memory and waiting for memory to send back the next instruction byte or bytes.
  - **Except in the case of JMP and CALL instructions, where the queue must be dumped and then reloaded starting from a new address,** this pre-fetch and queue scheme greatly speeds up processing.
  - Fetching the next instruction while the current instruction executes is called **pipelining.**

Register organization:

- 8086 has a powerful set of registers known as general purpose registers and special purpose registers.
- All of them are 16-bit registers.
- **General purpose registers:**
  - These registers can be used as either 8-bit registers or 16-bit registers.
  - They may be either used for holding data, variables and intermediate results temporarily or for other purposes like a counter or for storing offset address for some particular addressing modes etc.
- **Special purpose registers:**
  - These registers are used as segment registers, pointers, index registers or as offset storage registers for particular addressing modes.
- The 8086 registers are classified into the following types:
  - General Data Registers
  - Segment Registers
  - Pointers and Index Registers
  - Flag Register

1. General Purpose Registers



General Purpose Registers

- AX
- It is Accumulator .
- 16 bits and is divided into two 8-bit registers AH and AL to perform 8-bit instructions.
- used for arithmetical and logical instructions.
- Example:
  - ADD AX,02H
  - ADD AL,40H

- BX
- It is Base register.
- 16 bits and is divided into two 8-bit registers BH and BL to perform 8-bit instructions.
- Mainly used for:
  - Address calculation
  - Indexed addressing
  - Holding offsets
- Example:
  - MOV BL,50H
  - MOV BH,02H

  these two commands effectively set the BX register to 0250H ((BH:BL)).

What is offset

- **Offset : It tells where the data/instruction is located inside the segment**
- Physical Address = Segment × 16 + Offset, where
  - Segment → Starting point (base)
  - Offset → Position inside that segment

  Example:

  DS = 2000H (Segment)

  BX = 0040H (Offset) Then,

  Physical Address = 2000H × 10H + 0040H

  = 20000H + 0040H

  = **20040H**

General Purpose Registers

- CX
- counter register
- 16 bits and is divided into two 8-bit registers CH and CL to perform 8-bit instructions.
- Used for Loop instructions, Shift and rotate instructions.
- Example:
  - MOV CX,0005

- DX
- Data register
- 16 bits register and is divided into two 8-bit registers DH and DL to perform 8-bit instructions.
- used in multiplication an input/output port addressing.
- Example: MUL BX
  - DX:AX = AX × BX
  - AX contains one operand
  - BX contains the other operand
  - Result can be 32 bits
  - Lower 16 bits → AX
  - **Upper 16 bits → DX**

Example of use of DX

- MUL BX
- AX = FFFFH
- BX = 0002H
- DX:AX = 1FFFEH
- DX = 0001H
- AX = FFFEH
- CF (Carry Flag) and OF (Overflow Flag):
- Set to 1 if DX ≠ 0
- Cleared if DX = 0
- Other flags → undefined

SPECIAL PURPOSE REGISTERS

- SP
  - Stack pointer
  - 16 bit registers
  - It points to the topmost element of stack.
  - SP is **initialized by the programmer or OS**
  - Common initial values:FFFEH (even address)Depends on stack size
- BP
  - Base pointer.
  - 16 bit registers.
  - It's prime use is accessing parameters passed via the stack.
  - Especially important in:
    - Subroutines
    - Procedures
    - Function calls
  - SP keeps changing during: PUSH POP, BP remains fixed, so parameters can be accessed reliably.

SP → Elevator position (moves up and down)
BP → Floor reference point (fixed)

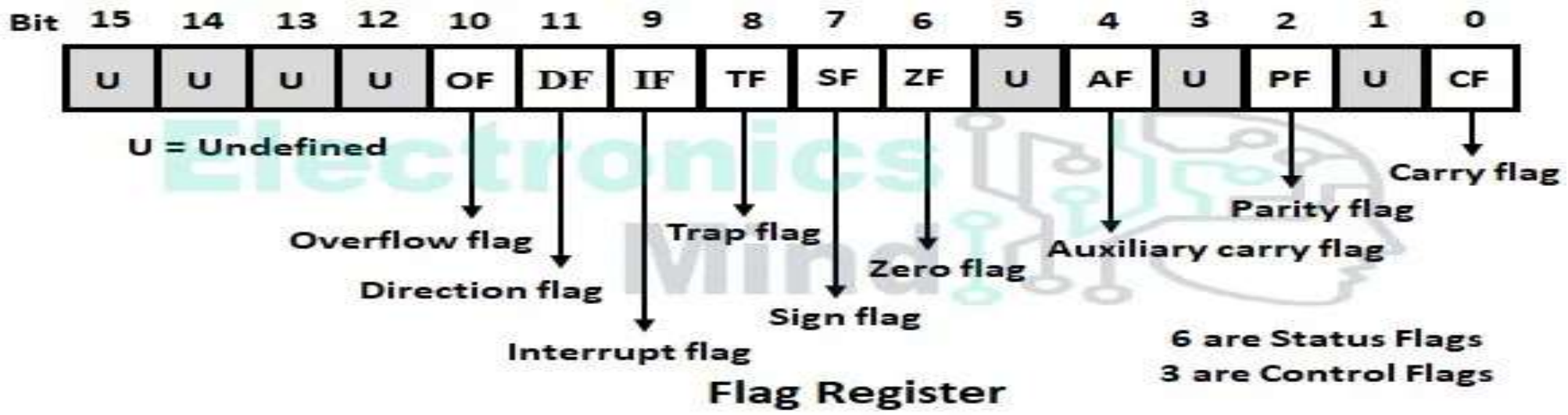
SPECIAL PURPOSE REGISTERS

- SI
  - Source index register
  - 16-bit registers.
  - Used to point the data and source in some string related operations.
  - It's offset is relative to data segment.
- Typical Uses of SI:
  - Reading data from memory
  - Copying strings
  - Comparing strings

- DI
  - Destination index register.
  - 16-bit registers.
  - used to point the destination in some string related operations.
  - It's offset is relative to extra segment.
  - Typical Uses of DI:
  - Writing data to memory
  - Writing String copy operations to destination
  - String comparison, result in Di

SI is a 16-bit source index register used to point to source data in the data segment during string operations. DI is a 16-bit destination index register used to point to the destination location in the extra segment. Both registers are mainly used in string manipulation instructions in the 8086 microprocessor.

Flag Registers



- The flag register of 8086 is a 16-bit register that contains 16 flip-flops. So, it can store a maximum of 16-bit of data.
- Out of 16-bits, 9-bits are used as flags as shown in the above figure.
- These nine flags are divided into two parts as
  - status flags and control flags.
  - Status flags tell **what happened,** control flags tell **what to do next.**

Status Flags

- When the microprocessor performs an arithmetic or logical operation in ALU, then depending upon the status of the result, the microprocessor will store corresponding status bits 0 or 1 in status flags. The status flags are,
  - Carry flag (CF)
  - Parity flag (PF)
  - Auxiliary carry flag (AF)
  - Zero flag (ZF)
  - Sign flag (SF), and
  - Overflow flag (OF).

Status Flags

- **Carry Flag (CF) :**
  - When the microprocessor performs the **addition** of two 8 or 16-bit numbers the result obtained in ALU will be a maximum of 9 or 17 bit.
  - **The last carry generated is directly copied into carry flag.** Similarly, when the microprocessor performs subtraction of (x – y) of two 8 or 16-bit numbers then,
  - **If x ≥ y, then the additional borrow required to perform subtraction is zero, so CF = 0.**
  - **If x ≤ y, then additional borrow is required.**
  - **So CF = 1** and the result of the subtraction (x – y) will be a negative number represented using 2's complement method.

Set to 1 if there is a carry or borrow

- **Parity Flag (PF) :**
  - When the **microprocessor performs an arithmetic or logical operation** in ALU the parity status of only 8 LSBs (least significant bit) of 8 or 16-bit result is stored into parity flag.
  - If the number of ones in **8 LSBs of result is even i.e., 0, 2, 4, 8, etc, then PF = 1.**
  - If the number of ones in 8 LSBs of **result is odd i.e., 1, 3, 5, 7, etc then PF = 0.**

Is the number of 1s even? Ie.. Even ones → PF ON

Status Flags

- Auxiliary Carry Flag (AF) :
  - When the microprocessor performs the addition of two 8 or 16-bit numbers, the carry generated after the addition of 4 LSBs (least significant bit) is copied into the AF flag.
  - Similarly, when the microprocessor performs subtraction of two 8 or 16-bit numbers, the borrow required to perform subtraction of 4 LSBs is copied into the AF flag.
  - This flag cannot be accessed by the programmer, and it is used in representing a decimal number in binary code.

Used in BCD (decimal) arithmetic

Mostly internal; programmers rarely use it

- Zero Flag (ZF) :
  - When the microprocessor performs an arithmetic or logical operation in ALU and all the 8 LSBs or 16 LSBs of the result are zero, then ZF = 1.
  - If 8 or 16-bit result in ALU is non-zero, then ZF = 0.
  - This flag is also set whenever the result of a certain register becomes zero following an operation.

Is the result zero?

Status Flags

- Sign Flag (SF) :
  - When the microprocessor performs an arithmetic or logical operation of an 8-bit number, the MSB of the result (D7 bit) is directly copied into the sign flag.
  - Similarly, when the microprocessor performs an arithmetic or logical operation of a 16-bit number, MSBs of result (D15 bit) is directly copied into the sign flag.
  - If the sign flag is set, the result will be negative otherwise the result will be positive.

Is the result negative?

- Overflow Flag (OF) :
  - When the microprocessor performs an arithmetic operation of signed binary numbers and if the result is within the range then OF = 0. If the signed result is out of range, then OF = 1. The range of 8 bit signed numbers is +7FH to -80H, and the range of 16 bit signed numbers is +7FFFH to -8000H.
  - Example for CF, PF, AF, ZF, SF, OF: The below shows the status of all status flags after performing the addition and subtraction operation.

Set when result is too big or too small for signed range

Control Flags

- Depending upon the value of the controlling flag bit, the microprocessor will control a particular operation i.e., these flags are used to control certain operations. There are three controlling flags namely,
  - Interrupt flag (IF),
  - Trap flag (TP), and
  - Direction flag (DF)

Control Flags

- Interrupt Flag (IF) :
  - Interrupt flag is used to **enable or disable the hardware interrupt pin INTR**. If the **interrupt flag is made 1 by giving the instruction STI** (Set Interrupt flag), then INTR is enabled. **If the interrupt flag is made zero by giving the instruction CLI** (Clear interrupt flag), then INTR interrupt is disabled.
- Trap Flag :
  - If the **trap** flag is made **zero,** then the **microprocessor** will **execute** the **complete program** in **one operation** and it is called a free-running operation. If the **trap** flag is made **one,** then the **microprocessor** will **execute** the **program** in **single-stepping mode.** Single stepping mode is used to detect the errors in the program i.e., software debugging.
- Direction Flag (DF) :
  - String instructions are used for a block or chain of data, MOV SB/MOV SW instruction is used to transfer a block of bytes or words from source memory location to destination memory location.
  - The direction flag (DF) controls the direction of string operations. An operation is performed from the lower address to the higher address if the flag is reset (is 0) or from the higher address to the lower address if the flag is set (is 1).
  - Flag 0 → forward (left → right)
  - Flag1 → backward (right → left)

Moral..

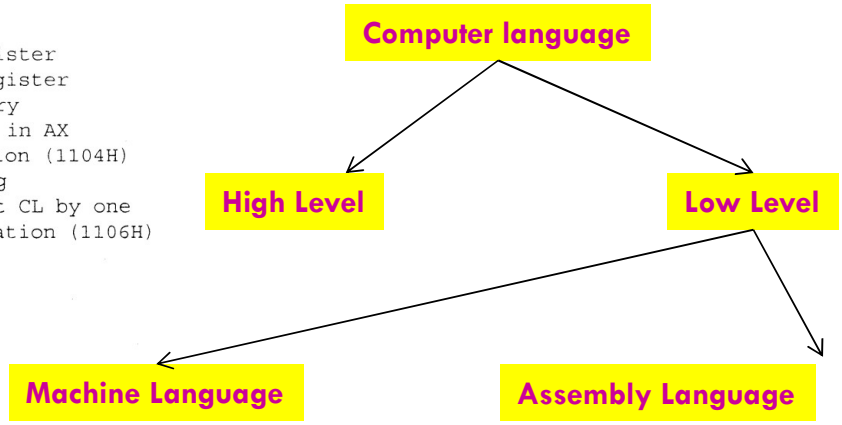

```
;PROGRAM TO ADD TWO 16-BIT DATA (METHOD-1)

DATA SEGMENT            ;Assembler directive

        ORG 1104H       ;Assembler directive
        SUM DW 0        ;Assembler directive
        CARRY DB 0      ;Assembler directive

DATA ENDS               ;Assembler directive

CODE SEGMENT            ;Assembler directive

        ASSUME CS:CODE  ;Assembler directive
        ASSUME DS:DATA  ;Assembler directive
        ORG 1000H       ;Assembler directive

        MOV AX,205AH    ;Load the first data in AX register
        MOV BX,40EDH    ;Load the second data in BX register
        MOV CL,00H      ;Clear the CL register for carry
        ADD AX,BX       ;Add the two data, sum will be in AX
        MOV SUM,AX      ;Store the sum in memory location (1104H)
        JNC AHEAD       ;Check the status of carry flag
        INC CL          ;If carry flag is set,increment CL by one
AHEAD:  MOV CARRY,CL    ;Store the carry in memory location (1106H)
        HLT

CODE ENDS               ;Assembler directive
END                     ;Assembler directive
```

Program

A set of instructions written to solve a problem.

Instruction

Directions which a microprocessor follows to execute a task or part of a task.



- Binary bits

- English Alphabets
- 'Mnemonics'
- Assembler Mnemonics → Machine Language

OPcodes

- https://www.javatpoint.com/instruction-set-of-8086

Debugger Basics

Debuggers help programmers to execute programs in a controlled manner, not only presenting to you the current state of the program, its memory, its register state, and so forth, but also allowing you to modify this state of the program while it is dynamically executing.

There are two types of debuggers based on the code that needs to be debugged: **source-level debuggers and machine-language debuggers.**

Source-level debuggers debug **programs at a high-level language level** and are popularly used by software developers to debug their applications. But unlike programs that have their high-level source code available for reference, we do not have the source code of malware when we debug them.

Instead, what we have are compiled binary executables at our disposal. To debug them, we use **machine language** binary debuggers like OllyDbg and IDA, which is the subject of our discussion here and which is what we mean here on when we refer to debuggers.

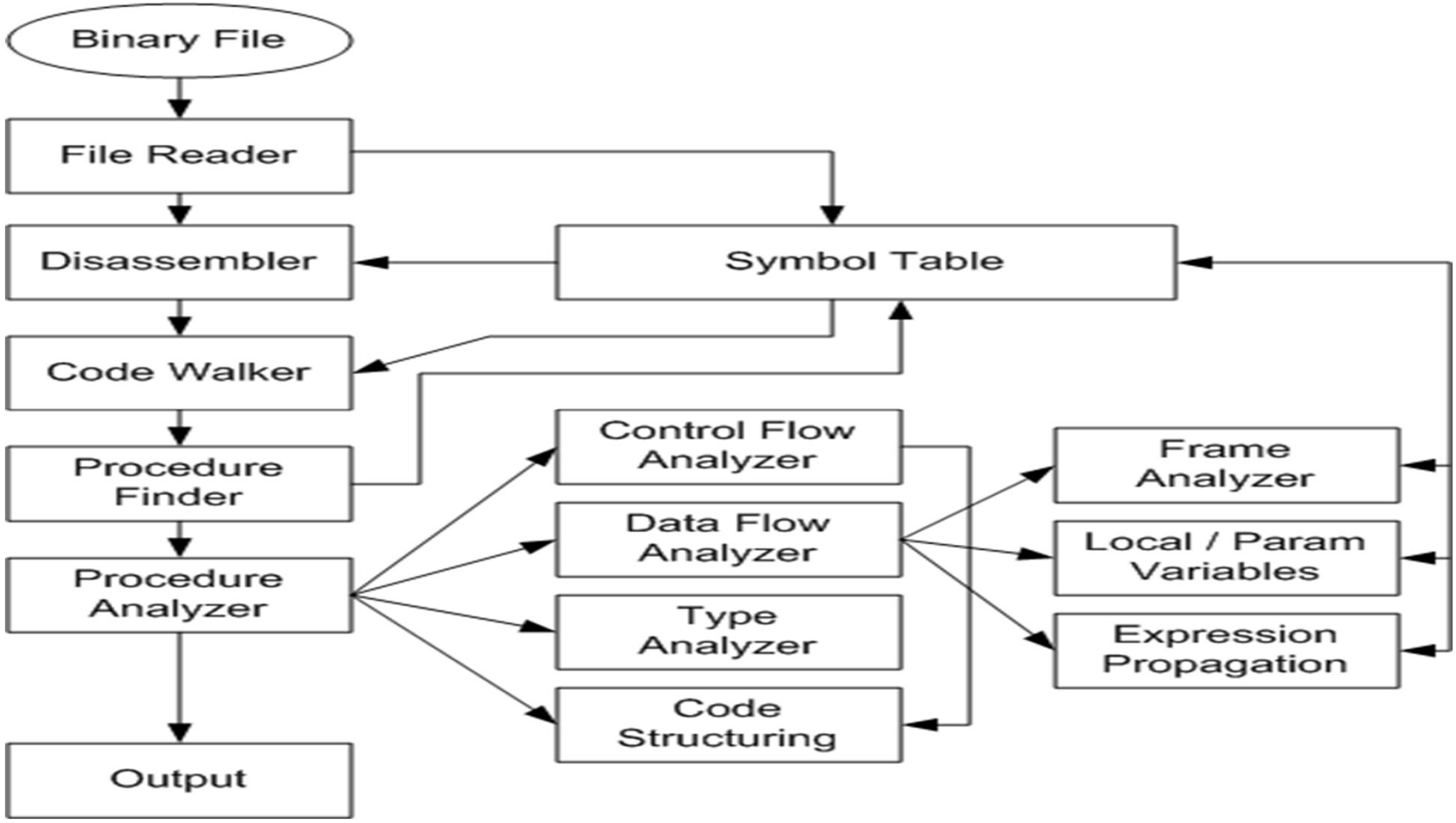
These debuggers allow us to debug the machine code by disassembling and presenting to us the machine code in assembly language format and allowing us to step and run through this code in a controlled manner.

Using a debugger, we can also change the execution flow of the malware as per our needs.

De-compiler

- A decompiler is a computer program that **translates an executable file to high-level source code.**
- It does therefore the opposite of a typical compiler, which translates a high-level language to a low-level language.
- While **disassemblers translate an executable into assembly language, de-compilers** go a step further and translate the code into a higher- level language such as C or Java, requiring more sophisticated techniques.
- **De-compilers** are usually **unable to perfectly reconstruct the original source code,** thus will frequently produce obfuscated code. Nonetheless, they remain an important tool in the reverse engineering of computer software.

De-Compiler Architecture



Design of De-Compiler

- De-compilers can be thought of as composed of a series of phases each of which contributes specific aspects of the overall de-compilation process.
- Loader:
  - The first de-compilation phase loads and parses the input machine code or intermediate language program's binary file format.
  - It should be able to discover basic facts about the input program, such as the architecture (Pentium, PowerPC, etc.) and the entry point.
  - In many cases, it should be able to find the equivalent of the main function of a C program, which is the start of the user written code.
  - This excludes the runtime initialization code, which should not be decompiled if possible. If available, the symbol tables and debug data are also loaded. The front end may be able to identify the libraries used even if they are linked with the code, this will provide library interfaces.
  - If it can determine the compiler or compilers used it may provide useful information in identifying code idioms

- Disassembly:
  - A disassembler converts a sequence of bytes that will potentially be executed by the processor, into a series of text lines that represent the operation performed by those bytes.
  - This is a very crude process, and it is prone to errors. It is based on the assumption that the byte sequence being disassembled represents some processor instruction ("code"). (The disassembler does not know: Where code really starts, Where code ends, Which bytes are data, **It just blindly decodes bytes**)
  - **If the compiler has put some data in the text section, the disassembler will try to convert that into instructions; even worse than that, the instruction stream may become de-synchronized, since many instructions have arguments that span more than one byte.**

- Idioms:
  - In computer programming, a programming idiom or code idiom is a **group of code fragments sharing an equivalent semantic role, which returns frequently across software projects often expressing a special feature of a recurring construct in one or more programming languages or libraries.**
  - Developers recognize programming idioms by **associating and giving meaning (semantic role) to one or more syntactical expressions within code snippets (code fragments).**
  - The idiom can be seen as a concept **underlying a pattern in code,** which is represented in implementation by contiguous or scattered code fragments.
  - These fragments are available in several programming languages, frameworks or even libraries.
  - A programming idiom's semantic role is a natural language expression of a simple task, algorithm, or data structure that is not a built-in feature in the programming language being used, or, conversely, the use of an unusual or notable feature that is built into a programming language.
  - Knowing the idioms associated with a programming language and how to use them is an important part of gaining fluency in that language and transferring knowledge in the form of analogies from one language or framework to another.

- Program analysis:
  - Various program analyses can be applied to the IR (intermediate representation).
  - Expression propagation combines the semantics of several instructions into more complex expressions. For example,
  - For example,

```
mov   eax,[ebx+0x04]
add   eax,[ebx+0x08]
sub   [ebx+0x0C],eax
```

could result in the following IR after expression propagation:

```
m[ebx+12]  := m[ebx+12] - (m[ebx+4] + m[ebx+8]);
```

- Data flow analysis:
  - The places where register contents are defined and used must be traced using data flow analysis.
  - The same analysis can be applied to locations that are used for temporaries and local data. A different name can then be formed for each such connected set of value definitions and uses.
  - It is possible that the same local variable location was used for more than one variable in different parts of the original program.
  - Even worse it is possible for the data flow analysis to identify a path whereby a value may flow between two such uses even though it would never actually happen or matter in reality.
  - This may in bad cases lead to needing to define a location as a union of types. The decompiler may allow the user to explicitly break such unnatural dependencies which will lead to clearer code. This of course means a variable is potentially used without being initialized and so indicates a problem in the original program

- Type analysis:
  - A good machine code de-compiler will perform type analysis.
  - Here, the way registers or memory locations are used result in constraints on the possible type of the location.
  - For example, an **and** instruction implies that the operand is an integer; programs do not use such an operation on floating point values (except in special library code) or on pointers.
  - An **add** instruction results in three constraints, since the operands may be both integer, or one integer and one pointer (with integer and pointer results respectively; the third constraint comes from the ordering of the two operands when the types are different)

- Structuring:
- The penultimate de-compilation phase involves structuring of the IR into higher level constructs such as while loops and if/then/else conditional statements. For example, the machine code
- Code generation:
- The final phase is the generation of the high-level code in the back end of the de-compiler. Just as a compiler may have several back ends for generating machine code for different architectures, a de-compiler may have several back ends for generating high level code in different high-level languages.

Common Malware Characteristics at the Windows API Level

- Application programming interfaces (APIs) act as a **bridge between different software programs and platforms with different architectures.** They're a core tool in seamless system integration.
- Unfortunately, just like any other program or application, they're **not immune to cyberattacks.**
- An API call encompasses the process of:
  - A client application **submitting a request** to the API in question
  - The API **retrieving the requested data**
  - The **data getting delivered back to the client**

What makes them malicious?

- Malicious API calls are ones that target your system for evil purposes. They could include, for example:
  - **Malicious injections**: A perpetrator can inject a malicious script into a vulnerable API or insert malicious commands into an API message. (I.e., a command that deletes tables from a database.)
  - **Denial of service** (DoS or DDoS): An attack on a web API that attempts to overwhelm it. This may be via a sudden deluge of concurrent connections, or by sending/requesting high volumes of data in each request.
  - **Data theft (MiTM)**: A 'man in the middle' attack in which the perpetrator sits in the middle of a connection and intercepts communications

Detecting malicious API calls

- It's the **behavior** and **intent behind the API use** that makes a call malicious or friendly, this is not a simple task. What may be considered malicious behavior most of the time, could just be a confused user, and vice versa.
- To **detect** malicious API calls, the typical advice is to **create a sliding scale of behavior.**
- This would have user behavior that you expect on one end, and the obviously malicious on the other.
- The middle **grey area applies to the API calls that may or may not be malicious — working on a balance** of probability.

Sliding scale of behaviour.

- Benign :
  - Single API calls
  - Expected execution context
  - Signed binaries
  - Known publishers
- Middle: Grey Area (Most Important for Analysts)
  - Legitimate APIs
  - Slightly unusual behavior
  - Needs correlation
- Right Side: Clearly Malicious
  - Strong indicators
  - Rare in legitimate software
  - Clear malicious intent

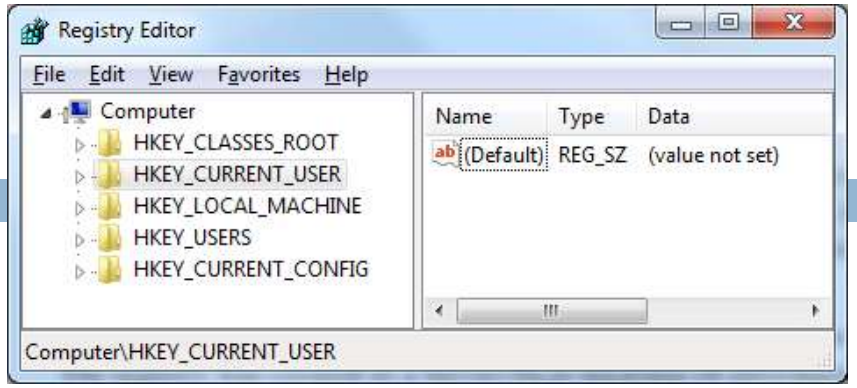
What kind of behavior should you be looking out for?

- **Privilege escalations** : Keep an eye out for accounts or users of an API that get more access rights than you expect. Any attempts at excess access — successful or otherwise — should be monitored. This behavior may be pointing toward malicious API calls.
- **High request rates** : I.e.,
  - **lots of API calls in a short amount of time** (particularly from one source). Too much traffic or high-volume requests can cause an API to slow down or stop working. It could result in the API suffering buffer overrun and so on.
  - High request rates are highly suspicious behavior when it comes to API calls. It can point to an ill-informed user, but typically it falls under unexpected behavior, which requires more scrutiny.
- **Mapping attempts:** There are reasons a developer might want to map an API. But, particularly when the API endpoints are internal or undocumented, it can suggest malicious intent. Namely, someone looking for weaknesses.

Registry Purpose

- Store **operating system and program configuration** settings
  - Desktop background, mouse preferences, etc.
- Malware uses the registry for **persistence**
  - Making malware re-start when the system reboots

Registry Terms



- **Root keys** These 5

- **Subkey** A folder within a folder
- **Key** A folder; can contain folders
  or values
- **Value entry** Two parts: name and data
- **Value** or **Data** The data stored in a registry entry
- REGEDIT Tool to view/edit the Registry

Root Keys

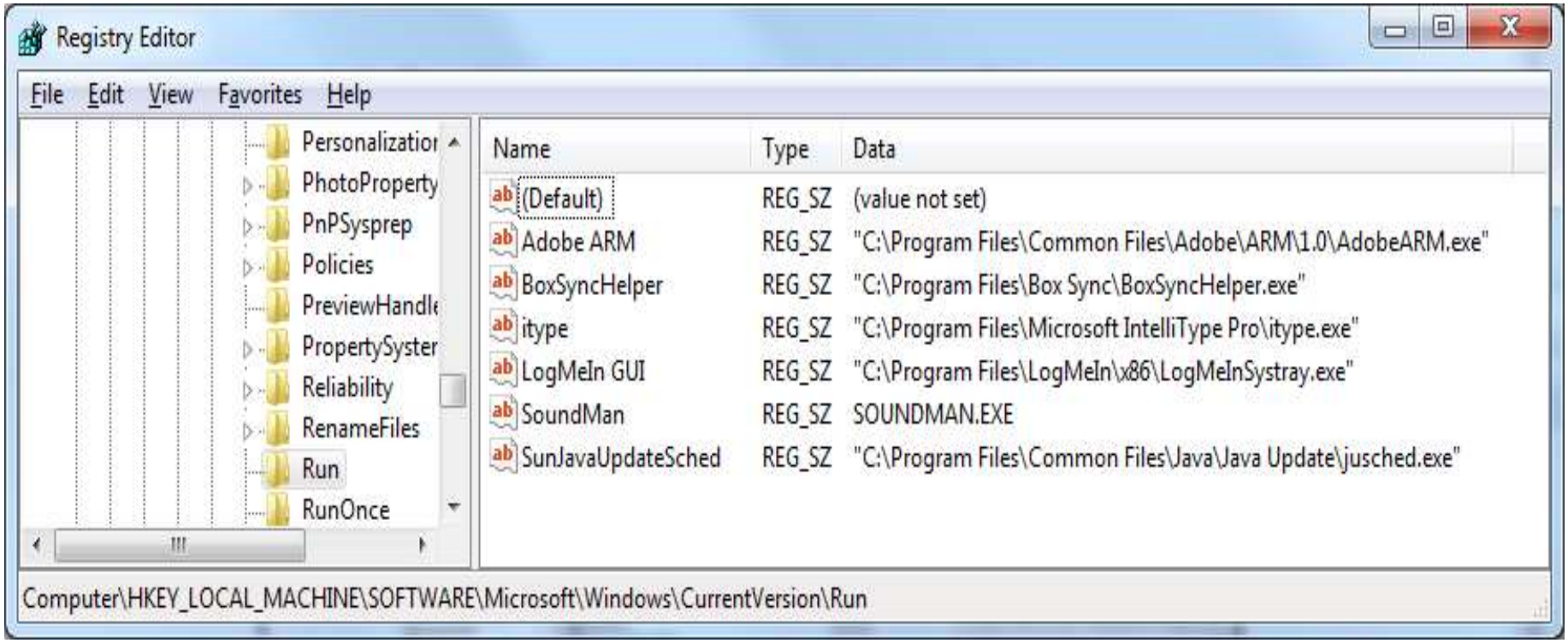
Registry Root Keys

The registry is split into the following five root keys:

- **HKEY_LOCAL_MACHINE (HKLM)**. Stores settings that are global to the local machine
- **HKEY_CURRENT_USER (HKCU)**. Stores settings specific to the current user
- **HKEY_CLASSES_ROOT**. Stores information defining types
- **HKEY_CURRENT_CONFIG**. Stores settings about the current hardware configuration, specifically differences between the current and the standard configuration
- **HKEY_USERS**. Defines settings for the default user, new users, and current users

Run Key

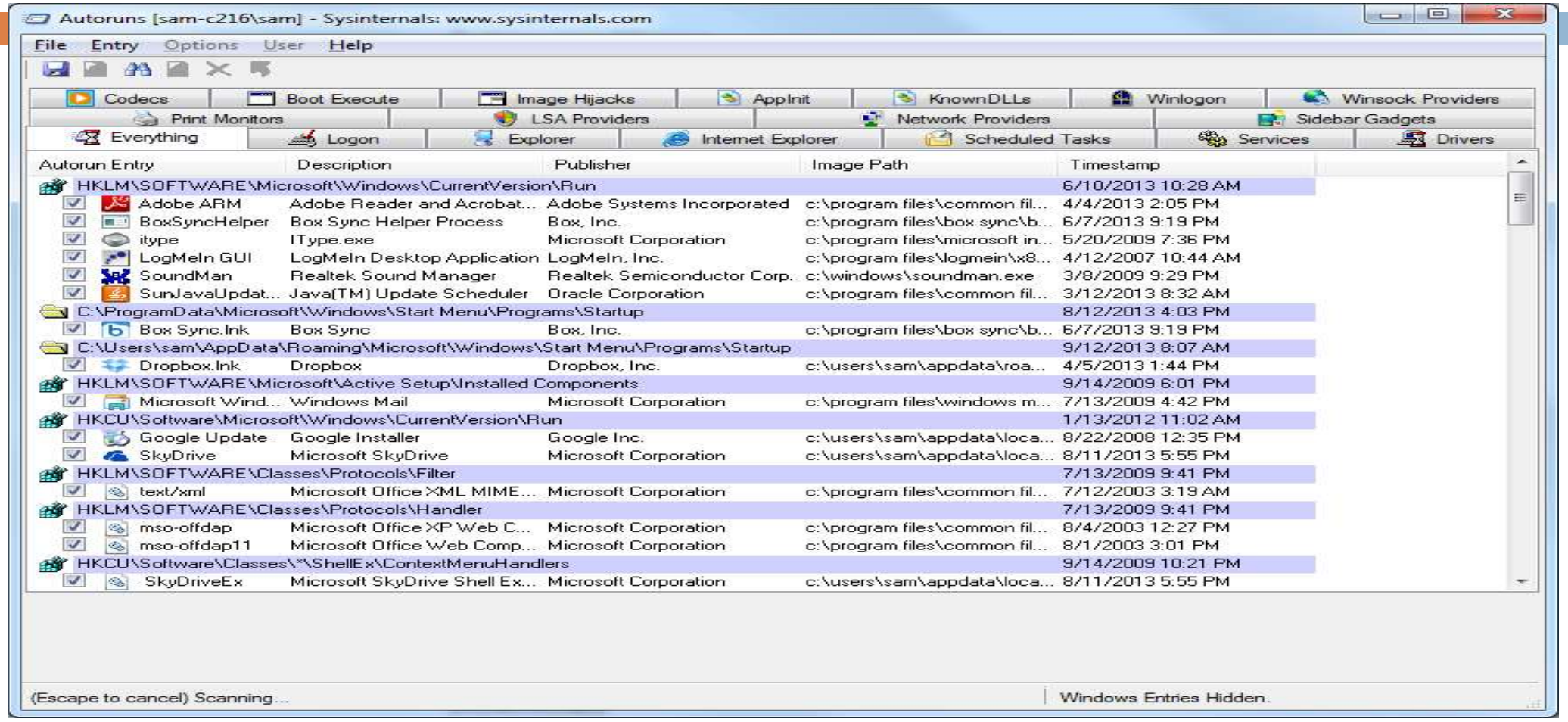
- **HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run**
  - Executables that start when a user logs on



Autoruns

- Sysinternals tool
- Lists code that will run automatically when system starts
  - Executables
  - DLLs loaded into IE and other programs
  - Drivers loaded into Kernel
  - It checks 25 to 30 registry locations
  - Won't necessarily find all automatically running code

Autoruns


Autoruns [sam-c216\sam] - Sysinternals: www.sysinternals.com
File Entry Options User Help
Codecs Boot Execute Image Hijacks AppInit KnownDLLs Winlogon Winsock Providers
Print Monitors LSA Providers Network Providers Sidebar Gadgets
Everything Logon Explorer Internet Explorer Scheduled Tasks Services Drivers
Autorun Entry Description Publisher Image Path Timestamp
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run 6/10/2013 10:28 AM
Adobe ARM Adobe Reader and Acrobat... Adobe Systems Incorporated c:\program files\common fil... 4/4/2013 2:05 PM
BoxSyncHelper Box Sync Helper Process Box, Inc. c:\program files\box sync\b... 6/7/2013 9:19 PM
itype IType.exe Microsoft Corporation c:\program files\microsoft in... 5/20/2009 7:36 PM
LogMeIn GUI LogMeIn Desktop Application LogMeIn, Inc. c:\program files\logmein\x8... 4/12/2007 10:44 AM
SoundMan Realtek Sound Manager Realtek Semiconductor Corp. c:\windows\soundman.exe 3/8/2009 9:29 PM
SunJavaUpdat... Java(TM) Update Scheduler Oracle Corporation c:\program files\common fil... 3/12/2013 8:32 AM
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup 8/12/2013 4:03 PM
Box Sync.lnk Box Sync Box, Inc. c:\program files\box sync\b... 6/7/2013 9:19 PM
C:\Users\sam\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup 9/12/2013 8:07 AM
Dropbox.lnk Dropbox Dropbox, Inc. c:\users\sam\appdata\roa... 4/5/2013 1:44 PM
HKLM\SOFTWARE\Microsoft\Active Setup\Installed Components 9/14/2009 6:01 PM
Microsoft Wind... Windows Mail Microsoft Corporation c:\program files\windows m... 7/13/2009 4:42 PM
HKCU\Software\Microsoft\Windows\CurrentVersion\Run 1/13/2012 11:02 AM
Google Update Google Installer Google Inc. c:\users\sam\appdata\loca... 8/22/2008 12:35 PM
SkyDrive Microsoft SkyDrive Microsoft Corporation c:\users\sam\appdata\loca... 8/11/2013 5:55 PM
HKLM\SOFTWARE\Classes\Protocols\Filter 7/13/2009 9:41 PM
text/xml Microsoft Office XML MIME... Microsoft Corporation c:\program files\common fil... 7/12/2003 3:19 AM
HKLM\SOFTWARE\Classes\Protocols\Handler 7/13/2009 9:41 PM
mso-offdap Microsoft Office XP Web C... Microsoft Corporation c:\program files\common fil... 8/4/2003 12:27 PM
mso-offdap11 Microsoft Office Web Comp... Microsoft Corporation c:\program files\common fil... 8/1/2003 3:01 PM
HKCU\Software\Classes*\ShellEx\ContextMenuHandlers 9/14/2009 10:21 PM
SkyDriveEx Microsoft SkyDrive Shell Ex... Microsoft Corporation c:\users\sam\appdata\loca... 8/11/2013 5:55 PM
(Escape to cancel) Scanning...
Windows Entries Hidden.

Tab	What it Shows	Malware Relevance
Everything	All autoruns	Full forensic view
Logon	Run / RunOnce keys	🔥 Most abused
Services	Windows services	Rootkits
Drivers	Kernel drivers	Advanced malware
Scheduled Tasks	Task-based persistence	Common in modern malware
Explorer	Shell extensions	Stealth persistence

Common Registry Functions

Commonly used by installers, drivers, configuration tools.

- RegOpenKeyEx
  - Opens a registry key for editing and querying
- RegSetValueEx
  - Adds a new value to the registry & sets its data
- RegGetValue
  - Returns the data for a value entry in the Registry
- Note: **Documentation will omit the trailing W (wide) or A (ASCII) character in a call like RegOpenKeyExW**

Ex, A, and W Suffixes

FUNCTION NAMING CONVENTIONS

When evaluating unfamiliar Windows functions, a few naming conventions are worth noting because they come up often and might confuse you if you don't recognize them. For example, you will often encounter function names with an `Ex` suffix, such as `CreateWindowEx`. When Microsoft updates a function and the new function is incompatible with the old one, Microsoft continues to support the old function. The new function is given the same name as the old function, with an added `Ex` suffix. Functions that have been significantly updated twice have two `Ex` suffixes in their names.

Many functions that take strings as parameters include an A or a W at the end of their names, such as `CreateDirectoryW`. This letter does *not* appear in the documentation for the function; it simply indicates that the function accepts a string parameter and that there are two different versions of the function: one for ASCII strings and one for wide character strings. Remember to drop the trailing `A` or `W` when searching for the function in the Microsoft documentation.

NETWORKING APIS

- A network API **enables communication between the network and applications, web browsers and databases.**
- APIs and databases can also use create, read, update and delete (CRUD) functions to store and modify data

CRUD function	HTTP function	Action	Use case
CREATE	POST	Configure a network remotely	Add a virtual LAN (VLAN)
READ	GET	List network devices through telemetry	List devices in a network remotely
UPDATE	PUT/PATCH	Modify a network config	Change a VLAN's name
DELETE	DELETE	Delete unused VLANs	Delete a VLAN

Common use cases for network APIs are the following

Use case	When to use APIs	Why
Action batches	You need to deploy a software update to 1,000 network devices. Use a single API request to do everything at once.	It's tedious to configure or update devices one by one, and APIs can help.
Telemetry	You need to see active devices remotely.	Using an API provides an easy way to view the devices, and you can filter the results with advanced features compared to the CLI.
Provisioning	You need to automate manual tasks, like configuring ports or load-balancing policies.	Avoid tedious CLI tasks for complex configs.

Common API used in Malware

- These APIs are frequently observed in malware samples because they help malware hide, survive, and evade analysis
- Encryption:
  - WinCrypt() (Windows Cryptographic API USED FOR HIDING PAYLOAD, obfuscation)
  - CryptAcquireContext() (Acquires a cryptographic provider handle)
  - CryptGenKey() (Generates a **random cryptographic key**)
  - CryptDeriveKey() (Derives a key from a password or hash)
  - CryptReleaseContext() (Cleans up cryptographic context)
- Anti-Analysis/VM
  - IsDebuggerPresent()
  - GetSystemInfo()
  - GlobalMemoryStatusEx()
  - GetVersion()
  - CreateToolhelp32Snapshot [Check if a process is running]

- Stealth:
  - VirtualAlloc
  - VirtualProtect
  - ReadProcessMemory
  - WriteProcessMemoryA/W
  - CreateRemoteThread
- Miscellaneous:
  - GetAsyncKeyState() -- Key logging
  - SetWindowsHookEx -- Key logging
  - GetForeGroundWindow -- Get running window name (or the website from a browser)
  - LoadLibrary() -- Import library
  - GetProcAddress() -- Import library
  - CreateToolhelp32Snapshot() -- List running processes
  - GetDC() -- ScreenshotBitBlt() -- ScreenshotInternetOpen(),
  - InternetOpenUrl(), InternetReadFile(), InternetWriteFile() -- Access the Internet
  - FindResource(), LoadResource(), LockResource() -- Access resources of the executable

Malware Techniques

- DLL Injection:
  - Execute an **arbitrary DLL** inside another process
  - Locate the process to inject the malicious DLL: CreateToolhelp32Snapshot, Process32First, Process32Next
  - Open the process: GetModuleHandle, GetProcAddress, OpenProcess
  - Write the path to the DLL inside the process: VirtualAllocEx, WriteProcessMemory
  - Create a thread in the process that will load the malicious DLL: CreateRemoteThread, LoadLibrary
- Reflective DLL Injection:
  - **Load** a **malicious DLL** without calling normal Windows API calls.
  - The DLL is mapped inside a process, it will resolve the import addresses, fix the relocations and call the DllMain function

Run attacker-controlled code inside another legitimate process

- Thread Hijacking:
  - Find a thread from a process and make it load a malicious DLL
  - Find a target thread: CreateToolhelp32Snapshot, Thread32First, Thread32Next
  - Open the thread: OpenThread
  - Suspend the thread: SuspendThread
  - Write the path to the malicious DLL inside the victim process: VirtualAllocEx, WriteProcessMemory
  - Resume the thread loading the library: ResumeThread
- PE Injection:
  - Portable Execution Injection: The executable will be written in the memory of the victim process and it will be executed from there.

- Process Hollowing:
  - The malware will unmap the legitimate code from memory of the process and load a malicious binary
  - Create a new process: CreateProcess
  - Unmap the memory: ZwUnmapViewOfSection, NtUnmapViewOfSection
  - Write the malicious binary in the process memory: VirtualAllocEc, WriteProcessMemory
  - Set the entry point and execute: SetThreadContext, ResumeThread

How analysts use this knowledge

- From a reverse engineering perspective: These APIs are analysis landmarks
  - Set breakpoints here
  - Observe decision-making logic
- From a dynamic analysis perspective:
  - Correlate with:
  - ProcMon
  - API Monitor
  - Sandbox logs
- From a classification perspective:
  - Encryption + anti-analysis → evasive malware
  - Anti-VM only → sandbox-aware malware
  - Heavy crypto → packer / ransomware family

"Malware rarely invents new APIs. It misuses normal Windows APIs in suspicious ways. Your job is to identify intent, not just function names."

"One API tells you nothing. Ten APIs together tell you a story."